Technical Documentation – Simple Motion Sensor

Team Lumora | IoT Creation Category

i. Members of the team

Position	Name	Block (if applicable)
Adviser	*Prof. Paul Vincent Corsina, MSCS*	*Faculty*
Leader	*Adriaan M. Dimate*	*BSCS 3-A*
Members	*Alliana Denise Eclarinal*	*BSIT 2--B*
	Roxel John Ferrarera	*BSCS 1-*
	Vinz Illagan	*BSIT 4-*
	Patrick Briane San Jose	*BSIT 2-*

1. System Definition and Description

1.1 Overview

An Arduino-Based Motion Activated Welcome System is a simple but powerful Internet of Things (IoT) project designed to automatically greet people when they enter a specific area. It combines sensors, microcontrollers, and audio output to create an interactive and automated experience.

1.2 System Objectives

- To design and develop an automated motion detection system

  Create a system using an Arduino and PIR sensor that can accurately detect motion within a specified range.
- To implement an audio-based response upon motion detection

  Program the system to trigger and play a beep through a buzzer whenever motion is detected.
- To ensure reliable and efficient standalone operation

  Develop a system that functions independently without internet connectivity, providing consistent performance, low power consumption, and minimal false triggers.

1.4 Technology Stack

Layer	Technology / Tool	Purpose
Cover	*Carton box*	*Protective Enclosure*
Backend	*.ino-based c++ tool*	*Programming of code*
Hardware	*Arduino UNO board and sensors* *PIR Motion Sensor*	*Serves as the main controller* *Serves as the sensor for detecting movement*

2. Circuit Diagram

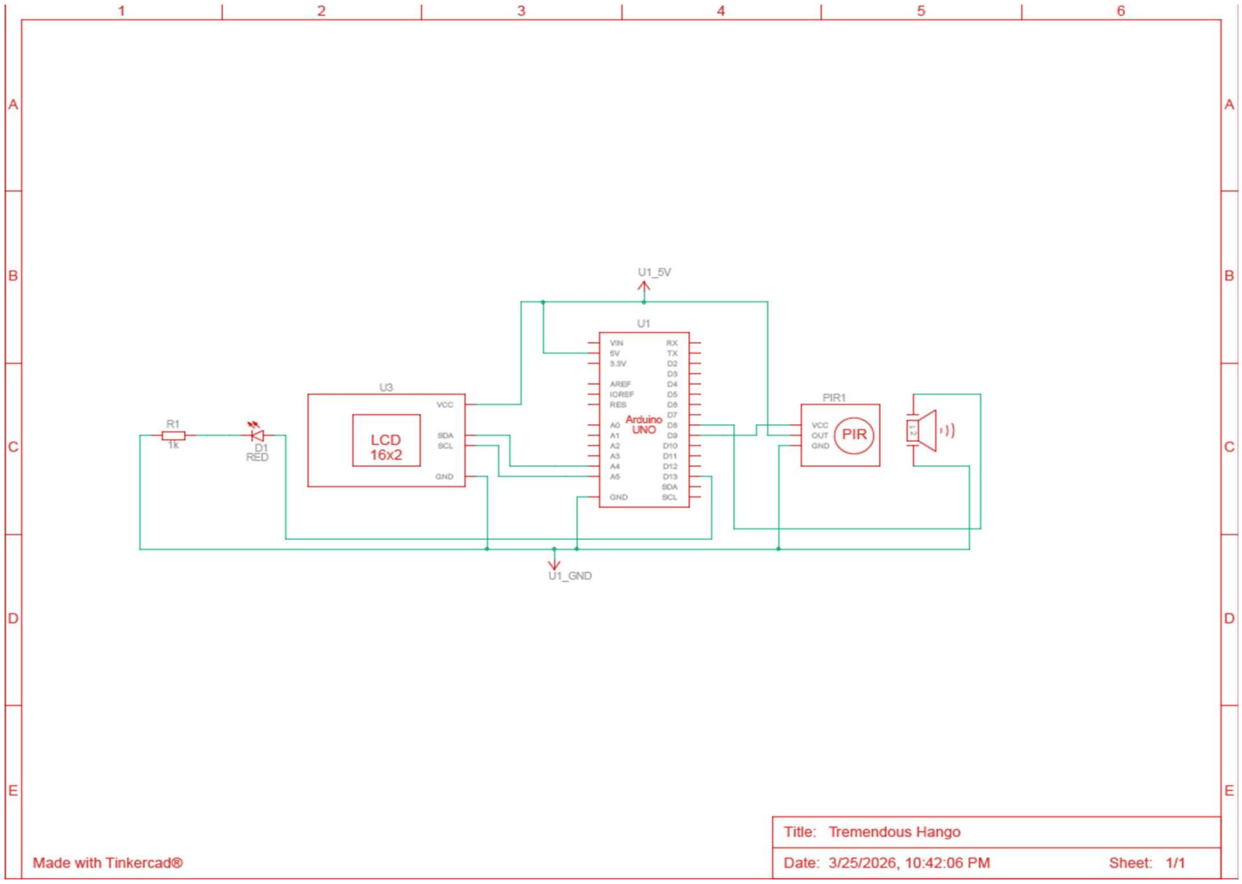
2.1 Schematic Overview

This circuit is an Arduino Uno–based motion detection system that uses a PIR sensor to detect movement within its range. When motion is detected, the sensor sends a digital signal to the Arduino, which serves as the central controller of the system. The Arduino continuously reads this input and determines whether motion is present or not.

Upon detecting motion, the Arduino activates both a buzzer and an LED to provide immediate audio and visual alerts. At the same time, it updates a 16x2 I2C LCD to display status messages such as “Motion Detected” or “No Motion.” The use of an I2C interface minimizes wiring complexity by requiring only two data lines (SDA and SCL) in addition to power connections.

All components in the circuit are powered through the Arduino’s 5V supply and share a common ground, ensuring stable and consistent operation. The system works in a continuous loop, constantly monitoring the PIR sensor output and responding accordingly. Due to its simplicity and effectiveness, this setup is well-suited for basic security systems or automated welcome applications in environments such as classrooms or entryways.

Figure 2.1. Schematic Diagram



2.2 Component List

Component	Quantity	Specification	Role in Circuit
Arduino Uno	*1*	*ATmega328P, 5V logic*	*Main controller that processes PIR input and controls output (LCD, buzzer, LED)*
PIR Motion Sensor	*1*	*5V operating voltage, digital output*	*Detects motion and sends HIGH/LOW signal to Arduino*
LCD 16x2 (I2C)	*1*	16x2 characters, I2C interface (SDA/SCL)	Displays system status messages (e.g., motion detected)
Piezzo Buzzer	*1*	5V Active/passive buzzer	Provides audible alert when motion is detected

LED (Blue)	*1*	Standard 5mm LED	Visual indicator for motion detection and debugging purposes
Resistor	*1*	1kΩ	Limits current to protect the LED
Jumper wires	*~20*	M2M/F2F	Connects components in the circuit
Power supply	*1*	5V USB Supply	Powers the entire system

3. Codebase Description

3.1 Repository Structure

The project follows a simple and modular embedded-system structure, as the implementation is contained within a single Arduino sketch.

```
project-root/
├── ccsweek/        — main Arduino source code (.ino file)
├── docs/           — documentation (design notes, diagrams)
└── README.md       — project overview and setup instructions
```

- **src/** contains the core logic for motion detection, display handling, and audio feedback.
- **docs/** stores supporting materials such as circuit diagrams and system descriptions.
- **README.md** provides instructions for setup, wiring, and usage.

The repository for the codebase is at **https://github.com/Thalanas110/ccs-week-lumora-iot**.

3.2 Module Overview

Provide a description of each major module or component in the codebase:

- **Sensor Module** - Handles input acquisition from the PIR motion sensor and determines whether motion is detected or not.

- **Display Module** - Manages the I2C LCD, updating messages based on system state (idle vs motion detected).

- **Audio Module** - Controls the piezo buzzer and plays a melody when motion is detected using timed note sequencing.

- **Control Logic Module** - Acts as the main decision-making unit, coordinating sensor input with display updates and audio output.

3.3 Data Flow

The system follows a real-time event-driven data flow:

PIR Sensor → Signal Read → Decision Logic → Output Actions → User Feedback

1. The **PIR motion sensor** continuously monitors movement and sends a HIGH/LOW signal to the microcontroller.
2. The system reads this signal using digitalRead() in the main loop.
3. The **control logic** evaluates the signal:
    - If motion is detected (HIGH), the system triggers output actions.
    - If no motion is detected (LOW), the system resets to idle state.
4. When motion is detected:
    - The **LED is activated**.
    - The **piezo buzzer plays a melody**.
    - The **LCD displays a welcome message**.
5. When motion ends:
    - The **LED turns off**.
    - The **buzzer stops**.
    - The **LCD reverts to standby mode**.

3.4 Key Algorithms / Logic

State Change Detection Algorithm:
Tracks transitions between motion and no-motion states using a sensorState variable. This prevents repeated triggering of outputs while motion remains continuously detected.

Non-blocking Melody Playback Algorithm:
Uses millis() timing instead of delays to play a sequence of notes. This allows the system to remain responsive while playing sound.

- Notes are stored in an array (melody[])
- Durations are stored in (noteDurations[])
- Timing is controlled using elapsed time comparison

Event-Driven Control Logic:
Implements conditional branching (if-else) based on sensor input to trigger appropriate system responses (LED, LCD, buzzer).

3.5 External Libraries and Dependencies

Library / Package	Version	Purpose
Wire.h	*Built in*	*Enables I2C communication between Arduino and LCD*
LiquidCrystal_I2C.h	*v1.1+*	*Controls the 16x2 LCD via I2C interface*

4. Codebase

4.1 Main Entry Point

The code implements a real-time motion-triggered welcome system using a PIR sensor, an I2C LCD, an LED, and a piezo buzzer. In the main loop, the Arduino continuously reads the PIR sensor state; when motion is detected (HIGH), it turns on the LED, updates the LCD to display a welcome message, and plays a short melody using a non-blocking timing mechanism based on millis(). A state variable (sensorState) ensures that the system only triggers actions once per motion event rather than repeatedly while motion persists. When no motion is detected (LOW), the system stops the sound, turns off the LED, and resets the LCD to a standby message. Overall, the program follows an event-driven design where sensor input directly controls visual and audio outputs in real time.

4.2 Codebase proper: ccsweek.cpp/ccsweek.ino

```
/*
 * PIR Motion Sensor with I2C LCD and Low-Tone Piezo
 * PIR Trigger: Pin 10
 * Piezo Buzzer: Pin 11
 * LCD SDA: A4
 * LCD SCL: A5
 */

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// for some dumb reason the lcd is at 0x27, so we initialize at 0x27 instead
LiquidCrystal_I2C lcd(0x27, 16, 2);

const int pirPin = 10;
```

```
const int ledPin = 13;
const int piezoPin = 11;

int sensorState = LOW;
int val = 0;

// main melody notes
#define NOTE_C4 262
#define NOTE_G3 196
#define NOTE_A3 220
#define NOTE_B3 247

int melody[] = {
  NOTE_C4, NOTE_G3, NOTE_G3, NOTE_A3,
  NOTE_G3, 0, NOTE_B3, NOTE_C4
};

int noteDurations[] = {
  4, 8, 8, 4,
  4, 4, 4, 4
};

int currentNote = 0;
unsigned long previousNoteTime = 0;

void setup() {
  pinMode(pirPin, INPUT);
  pinMode(ledPin, OUTPUT);
  pinMode(piezoPin, OUTPUT);

  Serial.begin(9600); // 30 ms delay for start warmups

  // Initialize the LCD and turn on the backlight
  lcd.init();
  lcd.backlight();

  Serial.println("System Ready.");
```

```
  // Print the ready state
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Detecting Motion"); // trimmed to 16 chars
}

void loop() {
  val = digitalRead(pirPin);

  if (val == HIGH) {
    digitalWrite(ledPin, HIGH);

    if (sensorState == LOW) {
      Serial.println("Motion detected!");
      toneSong();
      updateMovementDisplay();   // runs continuously while motion is detected
      delay(100);
      sensorState = HIGH;
    }
  }
  else {
    digitalWrite(ledPin, LOW);

    noSong();

    if (sensorState == HIGH) {
      Serial.println("Motion ended.");
      updateStillDisplay();
      sensorState = LOW;
    }
  }
}

// function that gets triggered when motion is detected.
void updateMovementDisplay() {
  lcd.setCursor(0, 0);
```

```
  lcd.print("MOVEMENT FOUND!!");
  lcd.setCursor(0, 1);
  lcd.print("Welcome, GCians!");
}

// when movement is no longer detected afterwards
void updateStillDisplay() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Detecting Motion"); // replace with something much better
}

// function for song.
// replace the tone with the necessary tone.
// gagawin ni aliana
void toneSong() {
  unsigned long currentTime = millis();
  int noteDuration = 1000 / noteDurations[currentNote];
  int pauseBetweenNotes = noteDuration * 1.30; // standard 30% gap between notes

  if (currentTime - previousNoteTime >= pauseBetweenNotes) {
    previousNoteTime = currentTime;

    if (melody[currentNote] != 0) {
      tone(piezoPin, melody[currentNote], noteDuration);
    }
    else {
      noTone(piezoPin); // explicit rest on 0-frequency notes
    }

    currentNote++;
    if (currentNote >= 8) {
      currentNote = 0;
    }
  }
}
```

```
void noSong() {
  // Stop the tone when motion ends
  noTone(piezoPin);
}
```

5. Installation Plan and Mounting

5.1 Prerequisites

- **Operating System** — Not required (system is preloaded)
- **Runtime / SDK** — Not required during deployment
- **Hardware Requirement** — Pre-assembled Arduino-based motion detection unit
  Battery pack (power source), 3M Double-Sided Tape (for mounting)

5.2 Environment Setup

No environment setup is required.

- The system is **pre-programmed and preloaded** with the necessary firmware.
- No installation of software, cloning of repositories, or dependency setup is needed.

5.3 Hardware Assembly and Mounting

No low-level assembly is required.

- All components (PIR sensor, LCD, buzzer, LED, and microcontroller) are **already pre-assembled and wired**.
- The device can be directly deployed.
- Mount the device using **3M double-sided tape** on the desired surface (e.g., the entrance to Rm510).
  - **A total of 6-8 strips will be used.**
  - **<u>No screws will be used to stick this to the wall, and no drilling will be used either.</u>**
- Power the system using:
  - **Battery pack/powerbank** for standalone operation

5.4 Configuration

No additional configuration is required.

- All parameters (pin assignments, LCD address, system logic) are **pre-configured in the firmware**.
- The system is ready for immediate use upon powering.

5.5 Running the System

To run the system:

- Connect the **battery pack** to power the device
- The system will automatically initialize and begin operation

5.6 Verification

To confirm correct operation:

- **Startup Behavior**:
  - LCD displays: "Detecting Motion"
  - System is in standby mode
- **Motion Detection Test**:
  - Move in front of the sensor
  - Expected behavior:
    - LED turns ON
    - LCD displays:
      - "MOVEMENT FOUND!!"
      - "Welcome, GCians!"
    - Piezo buzzer plays melody
- **Idle State**:
  - LED remains OFF
  - LCD shows standby message